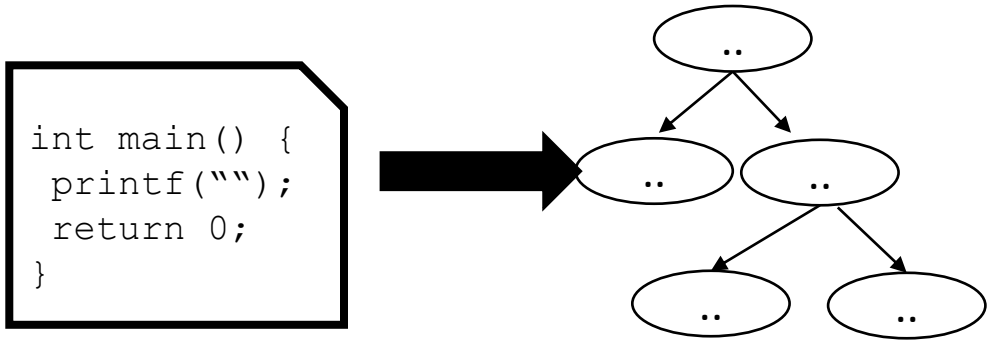
CSE110A: Compilers

Topics: Algorithms for Parsing

- *Syntactic Analysis continued*
  - *Top down parsing*
    - *Rewriting to avoid left recursion*
    - *Oracle Parsing*
    - *LL(1) Parser*



ORACULARITY:
MAKING A BACKTRACK FREE LL(1) PARSER USING FIRST, FOLLOW and FIRST+ SETS

New topic: Algorithms for parsing

One goal:

- Given a string *s* and a CFG *G,* determine if *G* can derive *s*

- We will do that by implicitly attempting to derive a parse tree for *S*

- Two different approaches, each with different trade-offs:
  - Top down
  - Bottom up

Top-down parsing

```
input: 2+3+4
```

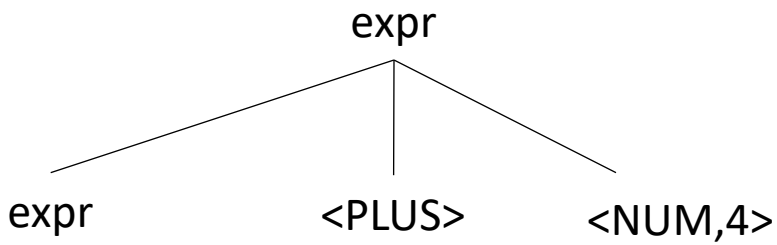
expr

Operator	Name	Productions
+	expr	: expr PLUS NUM | NUM

Top-down parsing

```
input: 2+3+4
```

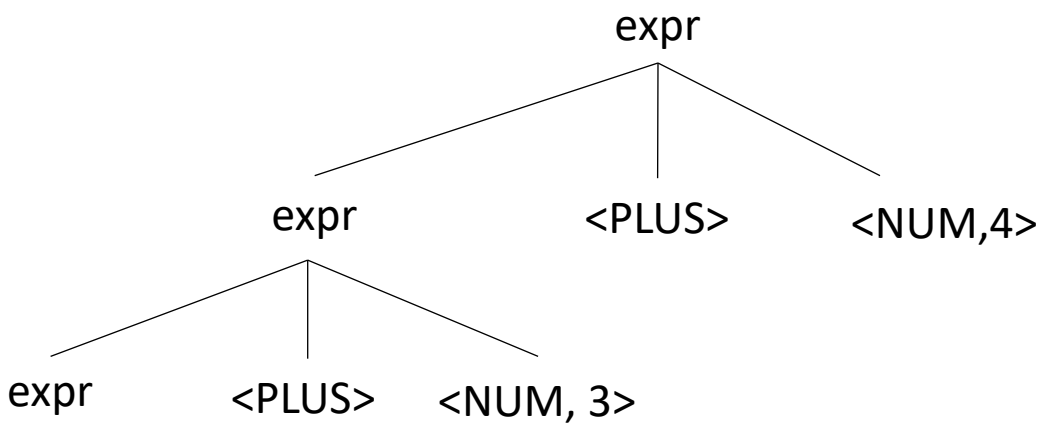
Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Top-down parsing

```
input: 2+3+4
```

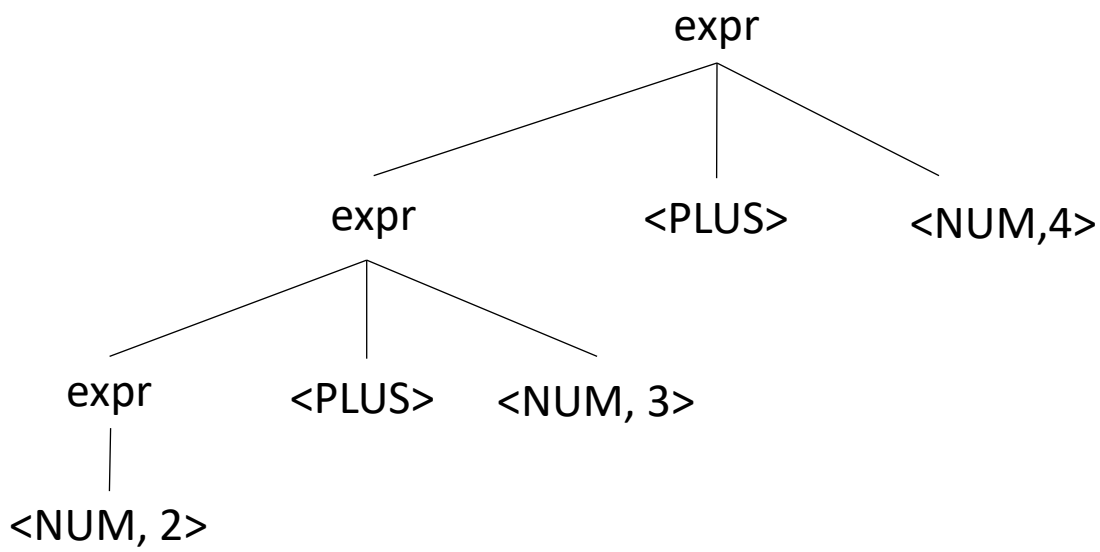
Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Top-down parsing

```
input: 2+3+4
```

Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Top-down parsing

Pros:

- Algorithm is simpler
- Oftentimes faster than bottom-up
- Better Error Messages

Cons:

- Not efficient on arbitrary grammars
- Many grammars need to be re-written
- More effective error recovery

Bottom-up parsing

```
input: 2+3+4
```

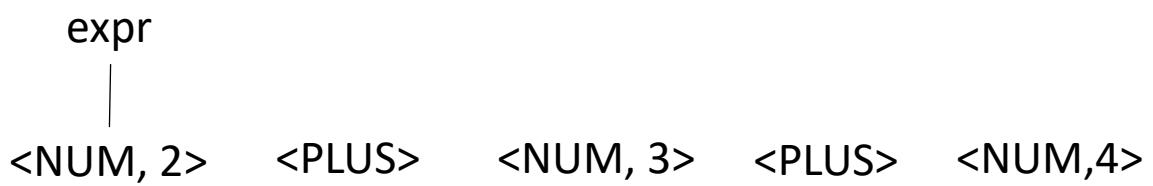
Operator	Name	Productions
+	expr	: expr PLUS NUM | NUM

<NUM, 2> <PLUS> <NUM, 3> <PLUS> <NUM,4>

Bottom-up parsing

```
input: 2+3+4
```

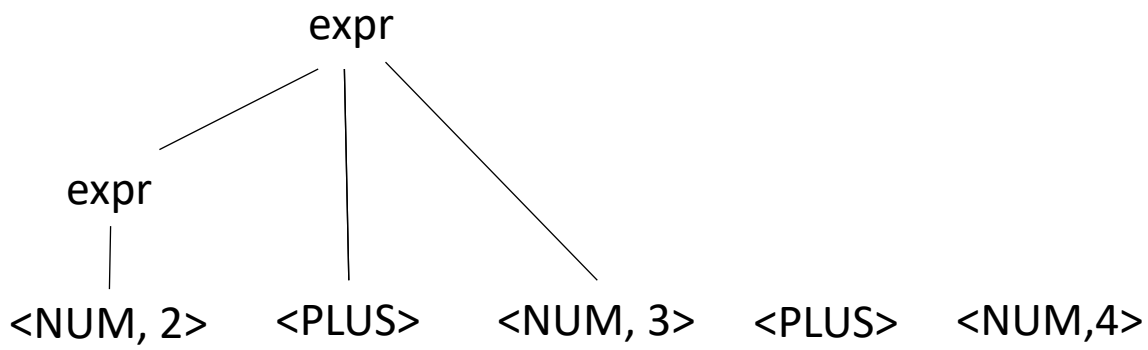
Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Bottom-up parsing

```
input: 2+3+4
```

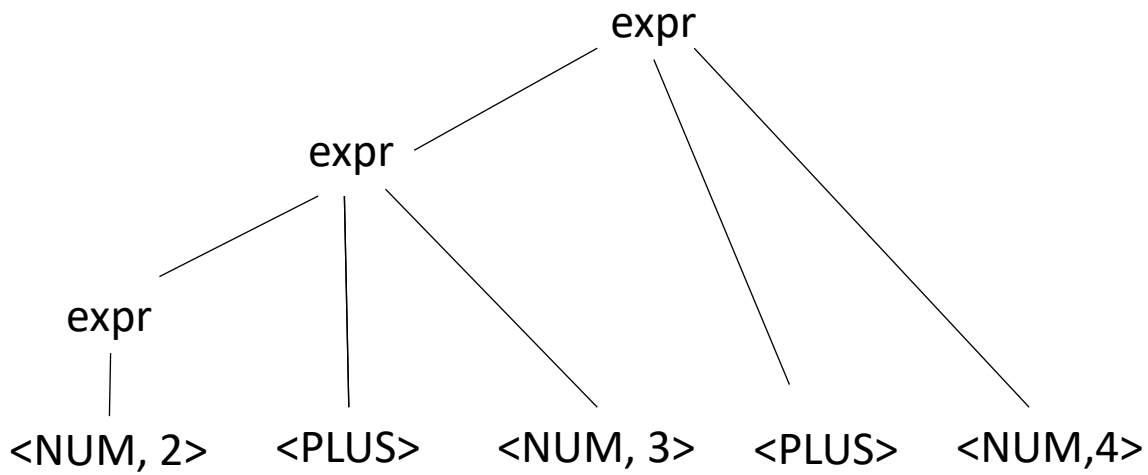
Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Bottom-up parsing

```
input: 2+3+4
```

Operator	Name	Productions
+	`expr`	`: expr PLUS NUM` `| NUM`



Bottom up

Pros:

- can handle grammars expressed more naturally
- can encode precedence and associativity even if grammar is ambiguous (with precedence and associativity specifications)

Cons:

- algorithm is complicated
- In some cases slower than top down

Let’s start with top-down parsing:

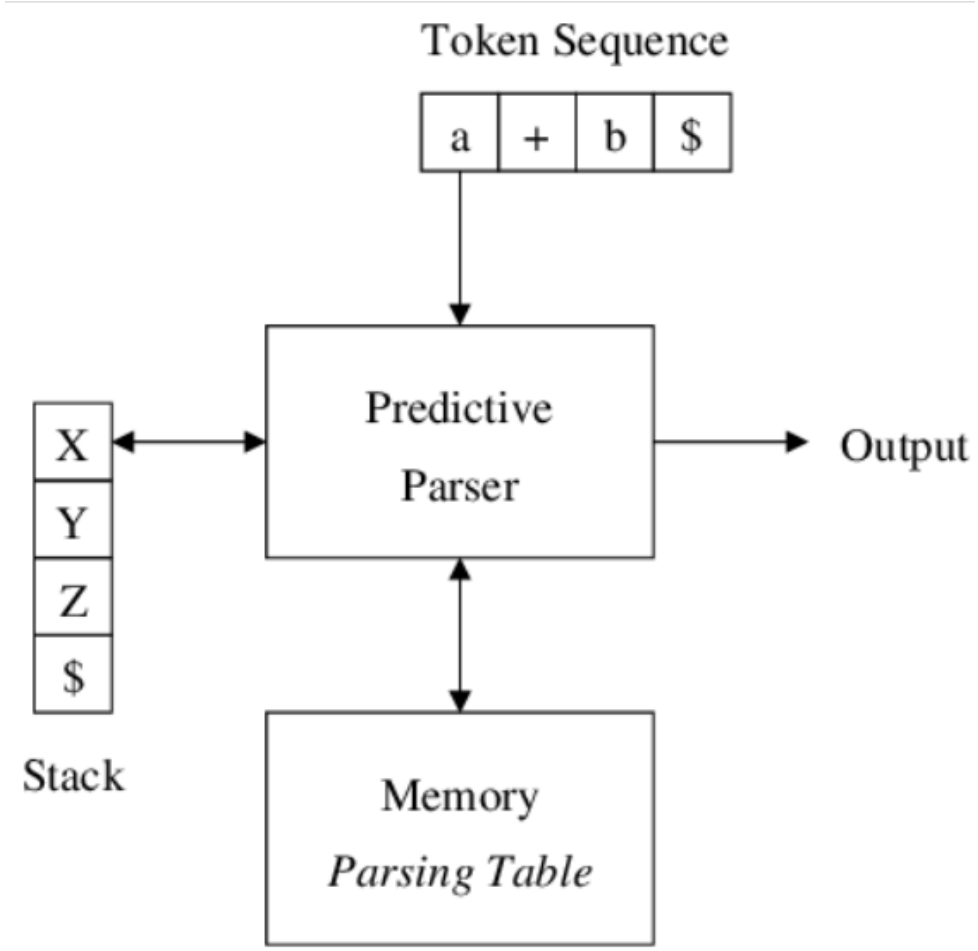
Two ways to implement the same idea:

1) LL(1) Push Down Automaton

2) Recursive Descent

Top down, Table Driven: LL(1) PDA Parser

Left to right processing with Leftmost derivation



Expand (Predict)
If top of stack is **non-terminal A**
And lookahead token is **a**
Consult table M[**A, a**]
Replace A in dysvk with chodr RHS
production (pushed in reverse order)

Match (Consume)
If top of stack is a **terminal a**
And lookahead token is also **a**
Pop stack
Advance input

Accept (or Error)
If top of stack is $
And input token is $
Parsing succeeds: Accept
Otherwise:
No table entry → **error**

3 Possible Actions

https://www.researchgate.net/figure/LL-Parser-Left-to-right-processing-with-leftmost-derivation_fig5_253273718

```
root = start symbol;    # Expr
focus = root;
push(None);
to_match = s.token();   # Read first token

while (true):  // Expand(Predict)
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1    # First symbol in rule

  else if (focus == to_match): // Match(Consume)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	[ ]

TOS ⬅

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Expanded Rule	Sentential Form
start	Expr

```
root = start symbol;   # Expr
focus = root;
push(None);
to_match = s.token();  # Read first token

while (true):
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1    # First symbol in rule

  else if (focus == to_match):
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Initialized State

Variable	Value
focus	Expr
to_match	"("
s.istring	*"a+b)*c"*
stack	[ ]

TOS ⬅

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Expanded Rule	Sentential Form
start	Expr

Initialize

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Variable	Value
focus	Expr -> Expr
to_match	"("
s.istring	*"a+b)*c"*
stack	[Op Unit]

TOS ⬅

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit

1st Iteration: Apply rule 1

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1                  // B1 is now '('

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Variable	Value
focus	Expr -> Unit
to_match	"("
s.istring	*"a+b)*c"*
stack	[Op Unit]

TOS ⬅

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit

2nd Iteration: Apply rule 2

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1        // B1 is now '('

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Variable	Value
focus	Unit -> "("
to_match	'('
's.istring	*+b)*c*
Stack	[Expr ')' Op Unit]

TOS ⬅

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit
3	'('Expr')' Op Unit

3rd Iteration: gobble the '('

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal, A)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1              // B1 is now '('

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Variable	Value
focus	"(" -> Expr
to_match	'(' -> (ID, "a")
's.istring	*+b)*c*
Stack	[')' Op Unit]

TOS ⬅

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit
3	Expr')' Op Unit

4th Iteration: consume '('

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	Op
to_match	'+'
s.istring	*b)*c*
stack	Unit ')' Op, Expr, None

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Expanded Rule #	Sentential Form
start	Expr
1	`Expr Op Unit`
2	Unit Op Unit
3	'(' Expr ')' Op Unit
1	'(' Expr Op Unit ')' Op Unit
2	'(' Unit Op Unit ')' Op Unit
4	'(' ID Op Unit ')' Op Unit

And so on...

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

BUT

What can go wrong if we don't have a magic Choice?

Variable	Value
focus	
to_match	
s.istring	
stack	

```
1: Expr   ::= Expr Op Unit
2:          |   Unit
3: Unit   ::= '(' Expr ')'
4:          |    ID
5: Op     ::= '+'
6:          |   '*'
```

Can we derive the string `(a+b)*c`

Expanded Rule	Sentential Form
start	Expr

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

What can go wrong?

Variable	Value
focus	
to_match	
s.istring	
stack	

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

Can we derive the string `(a+b)*c`

Expanded Rule	Sentential Form
start	Expr
2	Expr Op Unit
2	Expr Op Unit Op Unit
2	Expr Op Unit Op Unit Op Unit
2	Expr Op Unit ....

Infinite recursion, that is what!

Top down parsing does not handle left recursion

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

direct left recursion

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |    ID
6: Op        ::= '+'
7:           |   '*'
```

indirect left recursion

Top down parsing cannot handle either of these

Top down parsing does not handle left recursion

- In general, any CFG can be re-written without left recursion
  - However, the transformation may affect associativity
  - or increase the number of rules
  - but it is always possible

Eliminating direct left recursion

```
Fee ::= Fee “a”
    |   “b”
```

What does this grammar describe?

Eliminating direct left recursion

```
Fee ::= Fee “a”
    |   “b”
```

The grammar can be rewritten as

```
Fee  ::= “b” Fee2

Fee2 ::= “a” Fee2
     |    “”
```

Eliminating direct left recursion

In general, A and B can be any sequence of non-terminals and terminals

```
Fee ::= Fee A
    |   B
```

```
Fee  ::= B Fee2

Fee2 ::= A Fee2
     |   “”
```

Eliminating direct left recursion

```
1: Expr  ::= Expr Op Unit
2:       |   Unit
3: Unit  ::= '(' Expr ')'
4:       |    ID
5: Op    ::= '+'
6:       |   '*'
```

Lets do this one as an example:

```
Fee ::= Fee A
    |   B
```

```
Fee  ::= B Fee2

Fee2 ::= A Fee2
     |    ""
```

Eliminating direct left recursion

```
A = ?
B = ?
```

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

```
1: Expr   ::= ?
2: Expr2  ::= ?
3:        | ?
4: Unit   ::= '(' Expr ')'
5:        |    ID
6: Op     ::= '+'
7:        |   '*'
```

Lets do this one as an example:

```
Fee  ::= Fee A
     |   B
```

```
Fee   ::= B Fee2

Fee2  ::= A Fee2
      |    “”
```

Eliminating direct left recursion

```
A = Op Unit
B = Unit
```

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

```
1: Expr   ::= ?
2: Expr2  ::= ?
3:        | ?
4: Unit   ::= '(' Expr ')'
5:        |    ID
6: Op     ::= '+'
7:        |   '*'
```

Lets do this one as an example:

```
Fee  ::= Fee A
     |   B
```

```
Fee   ::= B Fee2

Fee2  ::= A Fee2
      |    ""
```

Eliminating direct left recursion

```
A = Op Unit
B = Unit
```

```
1: Expr  ::= Expr Op Unit
2:       |   Unit
3: Unit  ::= '(' Expr ')'
4:       |    ID
5: Op    ::= '+'
6:       |   '*'
```

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | ""
4: Unit  ::= '(' Expr ')'
5:       |    ID
6: Op    ::= '+'
7:       |   '*'
```

Lets do this one as an example:

```
Fee ::= Fee A
    |   B
```

```
Fee  ::= B Fee2

Fee2 ::= A Fee2
     |   ""
```

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:         |    ID
6: Op    ::= ‘+’
7:         |   ‘*’
```

Expanded Rule	Sentential Form
start	Expr

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	

How to handle this case?

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | ""
4: Unit  ::= '(' Expr ')'
5:         |   ID
6: Op    ::= '+'
7:         |   '*'
```

Expanded Rule	Sentential Form
start	Expr

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    if A == "": focus=pop(); continue; # ignore it
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

How to handle this case?

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | ""
4: Unit  ::= '(' Expr ')'
5:         |   ID
6: Op    ::= '+'
7:         |   '*'
```

Variable	Value
focus	
to_match	
s.istring	
stack	

Expanded Rule	Sentential Form
start	Expr

How about indirect left recursion?

```
1: Expr   ::= Expr Op Unit
2:        |   Unit
3: Unit   ::= '(' Expr ')'
4:        |    ID
5: Op     ::= '+'
6:        |   '*'
```

direct left recursion

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |    ID
6: Op        ::= '+'
7:           |   '*'
```

indirect left recursion

Top down parsing cannot handle either

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |   ID
6: Op        ::= '+'
7:           |   '*'
```

Identify indirect left left recursion

$$Expr\_base \rightarrow_{lhs} Expr\_op \rightarrow_{lhs} Expr\_base$$

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |    ID
6: Op        ::= '+'
7:           |   '*'
```

Identify indirect left left recursion

$$Expr\_base \rightarrow_{lhs} Expr\_op \rightarrow_{lhs} Expr\_base$$

Substitute indirect non-terminal closer to initial non-terminal

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |   ID
6: Op        ::= '+'
7:           |   '*'
```

```
1: Expr_base ::= Unit
2:           |   Expr_base Op Unit
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |   ID
6: Op        ::= '+'
7:           |   '*'
```

Identify indirect left left recursion

What to do with production rule 3?

$$Expr\_base \rightarrow_{lhs} Expr\_op \rightarrow_{lhs} Expr\_base$$

Substitute indirect non-terminal closer to initial non-terminal

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           |   Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |   ID
6: Op        ::= '+'
7:           |   '*'
```

```
1: Expr_base ::= Unit
2:           |   Expr_base Op Unit
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           |   ID
6: Op        ::= '+'
7:           |   '*'
```

Identify indirect left left recursion

What to do with production rule 3?
It may need to stay if another production rule references it!

$$Expr\_base \rightarrow_{lhs} Expr\_op \rightarrow_{lhs} Expr\_base$$

Substitute indirect non-terminal closer to initial non-terminal

It is always possible to eliminate left recursion

Back Tracking

(not a good thing)

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    cache_state();
    pick next rule (A ::= B1,B2,B3...BN);
    if B1 == "": focus=pop(); continue;
    push(BN... B3, B2);
    focus = B1

 else if (to_match == None and focus == None)
    Accept

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (we have a cached state)
     backtrack();

  else
    parser_error()
```

Keep track of what choices we've made

```
1: Expr  ::= ID Expr2
2: Expr2 ::= '+' Expr2
           |   ""
```

Can we match: "a"?

Expanded Rule	Sentential Form
start	Expr
1	ID Expr2
3	ID

Backtracking gets complicated...

- Do we need to backtrack?
  - In the general case, **yes**
  - In many useful cases, **no**

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    if B1 == "": focus=pop(); continue;
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	Expr2
to_match	None
s.istring	""
stack	None

```
1: Expr  ::= ID Expr2
2: Expr2 ::= '+' Expr2
3:       |   ""
```

Could we make a smarter choice here?

Can we match: "a"?

Expanded Rule	Sentential Form
start	Expr
1	ID Expr2

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

```
First sets:
1: {}
2: {}
3: {}
4: {}
5: {}
6: {}
7: {}
```

Example

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

```
First sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {“”}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

```
First sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {“”}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

We can use first sets to decide which rule to pick!

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

```
First sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {“”}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

Look-ahead to select Correct rule

We simply use to_match and compare it to the first sets for each choice

For example, Op and Unit

The Follow Set

Rules with “” in their First set *need special attention*

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

```
First sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {“”}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

```
Follow sets:
1: NA
2: NA
3: {}
4: NA
5: NA
6: NA
7: NA
```

We need to find the tokens that any string that follows the production can start with.

The Follow Set

Rules with “” in their First set *need special attention*

```
1: Expr   ::= Unit Expr2
2: Expr2  ::= Op Unit Expr2
3:          | “”
4: Unit   ::= ‘(‘ Expr ‘)’
5:          |    ID
6: Op     ::= ‘+’
7:          |   ‘*’
```

```
First sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {“”}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

```
Follow sets:
1: NA
2: NA
3: {None, ’)’}
4: NA
5: NA
6: NA
7: NA
```

We need to find all tokens that any string that follows a production containing ∈ In its first set can start with. In the example above Expr2 can be followed by what follows Expr or ‘)’.

Note that FIRST, FOLLOW, and FIRST+ are sets of terminals. But follow uses Non-terminals to calculate with transitive closure.

The First+ Set

The First+ set is conditional combination of First and Follow sets

$$\text{FIRST+}(\text{A} \rightarrow \beta) = \begin{cases} \text{FIRST}(\beta) & \text{if } \epsilon \in \text{FIRST}(\beta) \\ \text{FIRST}(\beta) \cup \text{FOLLOW}(\text{A}) & \text{otherwise} \end{cases}$$

where $\beta = X_1 X_2 \ldots X_n$ is a sequence of terminals/non-terminals

We only apply FOLLOW(A) if FIRST($\text{A} \rightarrow \beta$) has **ε** as a member.

This is pretty subtle

The First+ Set

The First+ set is the combination of First and Follow sets

```
                                First sets:    Follow sets:     First+ sets:
1: Expr  ::= Unit Expr2         1: {'(', ID}   1: NA            1: {'(', ID}
2: Expr2 ::= Op Unit Expr2      2: {'+', '*'}  2: NA            2: {'+', '*'}
3:       | ""                   3: {""}        3: {None, ')'}   3: {None, ')'}
4: Unit  ::= '(' Expr ')'       4: {'('}       4: NA            4: {'('}
5:       |    ID                5: {ID}        5: NA            5: {ID}
6: Op    ::= '+'                6: {'+'}       6: NA            6: {'+'}
7:       |   '*'                7: {'*'}       7: NA            7: {'*'}
```

Animation of First, Follow, and First+ Sets

https://youtu.be/JhEmguMmfLU

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:       |    ID
6: Op    ::= ‘+’
7:       |   ‘*’
```

First+ sets:

1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {None, ’)’}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}

For each non-terminal: if every production has a disjoint First+ set then we do not need any backtracking!

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:         |    ID
6: Op    ::= ‘+’
7:         |   ‘*’
```

```
First+ sets:
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {None, ’)’}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

For each non-terminal: if every production has a disjoint First+ set then we do not need any backtracking!

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | “”
4: Unit  ::= ‘(‘ Expr ‘)’
5:         |    ID
6: Op    ::= ‘+’
7:         |   ‘*’
```

First+ sets:

```
1: {‘(‘, ID}
2: {‘+’, ‘*’}
3: {None, ’)’}
4: {‘(‘}
5: {ID}
6: {‘+’}
7: {‘*’}
```

These grammars are called LL(1)

- L - scanning the input left to right
- L - left derivation
- 1 - how many look ahead symbols

They are also called predictive grammars

Many programming languages are LL(1)

None implies the end of the program.

*For each non-terminal: if every production has a **disjoint First+ set** then we do not need any backtracking!*

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID
2:          |   ID ‘[‘ Args ‘]’      # Array Indexing
3:          |   ID ‘(‘ Args ‘)’      # function calling
...
```

Sometimes the grammar needs to be refactored

```
                                    First
1: Factor ::= ID                    1: {}
2:          |   ID '[' Args ']'      2: {}
3:          |   ID '(' Args ')'      3: {}
...                                 ...
```

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID                    First
2:          |   ID '[' Args ']'      1: {ID}
3:          |   ID '(' Args ')'      2: {ID}
...                                  3: {ID}
                                     ...
```

We cannot select the next rule based on a single look ahead token!

Sometimes the grammar needs to be refactored

```
                                        First
1: Factor ::= ID                        1: {ID}
2:          |   ID ‘[‘ Args ‘]’          2: {ID}
3:          |   ID ‘(‘ Args ‘)’          3: {ID}
...                                     ...
```

We can refactor

```
                                              First
1: Factor      ::= ID Option_args             1: {}
2: Option_args ::= ‘[‘ Args ‘]’               2: {}
3:             |   ‘(‘ Args ‘)’               3: {}
4:             |   “”                         4: {}
```

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID                    First
2:          |   ID '[' Args ']'     1: {ID}
3:          |   ID '(' Args ')'     2: {ID}
...                                 3: {ID}
                                    ...
```

We can refactor

```
1: Factor       ::= ID Option_args      First
2: Option_args  ::= '[' Args ']'        1: {ID}
3:              |   '(' Args ')'        2: {'['}
4:              |   ""                  3: {'('}
                                        4: {""}   // We will need to compute the follow set
```

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID
2:          |   ID '[' Args ']'
3:          |   ID '(' Args ')'
...
```

```
First
1: {ID}
2: {ID}
3: {ID}
...
```

It is not always possible to rewrite any grammar into a predictive form, but many programming languages can be.

We can refactor

```
1: Factor      ::= ID Option_args
2: Option_args ::= '[' Args ']'
3:             |   '(' Args ')'
4:             |   ""
```

```
First
1: {ID}
2: {'['}
3: {'('}
4: {""}
```

// We will need to compute the follow set
// to deal with the empty string, and get
// First+

We now have a full top-down parsing algorithm!

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
  if (focus is a nonterminal)
    pick next rule (A ::= B1,B2,B3...BN);
    push(BN... B3, B2);
    focus = B1

  else if (focus == to_match)
    to_match = s.token()
    focus = pop()

  else if (to_match == None and focus == None)
    Accept
```

```
First+ sets:
1: {'(', ID}
2: {'+', '*'}
3: {None, ')'}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

First+ sets for each production rule

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | ""
4: Unit  ::= '(' Expr ')'
5:         |    ID
6: Op    ::= '+'
7:         |   '*'
```

input grammar, refactored to remove left recursion

To pick the next rule, compare `to_match` with the possible `first+` sets. Pick the rule whose `first+` set contains `to_match`.

If there is no such rule then it is a parsing error.